Práctica: Integración numérica distribuida mediante Java RMI

Computación Distribuida

1. Objetivo

El objetivo de esta práctica es desarrollar una aplicación distribuida utilizando **Java RMI (Remote Method Invocation)** que permita calcular, de forma paralela y distribuida, el valor aproximado de una integral definida mediante el método de **Monte Carlo**.

La aplicación estará formada por un proceso cliente y varios procesos servidores. Cada servidor realizará una parte independiente del cálculo y devolverá al cliente el resultado obtenido.

El cliente deberá crear **un hilo de ejecución por cada servidor**, de forma que las peticiones RMI puedan ejecutarse concurrentemente. De esta forma, la incorporación de nuevos servidores deberá permitir reducir el tiempo de ejecución, al menos de forma aproximada.

La práctica pretende familiarizar al alumno con:

- La arquitectura cliente/servidor de Java RMI.
- La definición y utilización de interfaces remotas.
- La implementación de objetos remotos.
- El registro RMI (`rmiregistry`).
- La utilización de hilos para ejecutar llamadas remotas en paralelo.
- La distribución equilibrada de un cálculo entre varios servidores.
- La medida y análisis del rendimiento de una aplicación distribuida.

2. Problema

Se desea calcular numéricamente la integral

$$I = \int_0^1 e^{-x^2}\, dx$$

mediante el método de Monte Carlo.

Para ello se generarán puntos

$$(x, y)$$

uniformemente distribuidos en el cuadrado

$$0 \leq x \leq 1, \qquad 0 \leq y \leq 1.$$

Un punto pertenece a la región situada bajo la curva

$$y = e^{-x^2}$$

si se cumple

$$y \leq e^{-x^2}.$$

Si se generan N puntos y M de ellos cumplen dicha condición, entonces la aproximación de la integral será

$$I \approx \frac{M}{N}.$$

El valor de referencia de la integral es aproximadamente

$$I \approx 0.7468241328.$$

Cuanto mayor sea el número de puntos generados, mayor será, en general, la precisión de la aproximación.

3. Arquitectura distribuida

La aplicación estará formada por un proceso cliente y varios procesos servidores.

3.1. Cliente

El cliente será responsable de:

1. Solicitar al usuario el número total de puntos que se desean generar.
2. Localizar los objetos remotos correspondientes a los servidores.
3. Dividir el número total de puntos entre los servidores disponibles.
4. Crear **tantos hilos como servidores**.
5. Asignar a cada hilo la realización de una llamada RMI a un servidor.
6. Ejecutar las llamadas de forma concurrente, de modo que el cliente no espere a que termine un servidor para comenzar la comunicación con otro.
7. Recibir y almacenar los resultados parciales.
8. Esperar a que finalicen todos los hilos.
9. Combinar los resultados obtenidos.
10. Calcular y mostrar la aproximación final de la integral.
11. Mostrar el tiempo total empleado en realizar el cálculo.

Importante: una implementación en la que el cliente invoque secuencialmente a los servidores no cumplirá el objetivo fundamental de esta práctica. Las llamadas a los diferentes servidores deberán poder estar ejecutándose simultáneamente.

El alumno podrá utilizar, entre otros mecanismos de Java, las clases `Thread` y/o `Runnable`.

3.2. Servidores

Cada servidor deberá:

1. Crear un objeto remoto que implemente la interfaz definida por la aplicación.
2. Registrar dicho objeto en el registro RMI.
3. Recibir del cliente el número de puntos que debe procesar.
4. Generar aleatoriamente los pares (x, y).
5. Comprobar para cada punto si se cumple $y \leq e^{-x^2}$.
6. Contabilizar el número de puntos que cumplen la condición.
7. Devolver al cliente el resultado de su cálculo.

Los servidores **no deberán comunicarse entre sí**. Toda la coordinación será realizada por el cliente.

4. Distribución del trabajo

Si el cliente solicita la generación de N puntos y existen S servidores, el trabajo deberá distribuirse entre ellos de forma equilibrada.

Todos los puntos deberán ser procesados, incluso cuando N no sea divisible entre S.

Por ejemplo, si

$$N = 10\,000\,003$$

y existen cuatro servidores, el cliente deberá distribuir los puntos de forma que la suma de los puntos asignados sea exactamente 10 000 003 y las cargas sean lo más equilibradas posible.

5. Paralelismo obligatorio

El aspecto fundamental de esta práctica es que el cálculo realizado por los servidores debe ejecutarse de forma concurrente.

Supóngase que existen cuatro servidores. El cliente deberá crear cuatro hilos:

$$\text{Hilo}_1, \quad \text{Hilo}_2, \quad \text{Hilo}_3, \quad \text{Hilo}_4.$$

Cada hilo realizará una llamada RMI independiente:

$$\text{Hilo}_i \longrightarrow \text{Servidor}_i.$$

De esta forma, mientras el servidor 1 está realizando su cálculo, el servidor 2, el servidor 3 y el servidor 4 podrán estar realizando simultáneamente sus respectivos cálculos.

Conceptualmente, la ejecución deberá aproximarse al siguiente esquema:

Servidor 1	Servidor 2	Servidor 3	Servidor 4
Cálculo	Cálculo	Cálculo	Cálculo

y no a una ejecución secuencial del tipo

$$\text{Servidor 1} \longrightarrow \text{Servidor 2} \longrightarrow \text{Servidor 3} \longrightarrow \text{Servidor 4}.$$

El objetivo es que la incorporación de servidores permita disminuir el tiempo necesario para realizar un número determinado de iteraciones.

6. Ejemplo

Supóngase que el cliente solicita

$$N = 100\,000\,000$$

puntos y que existen cuatro servidores.

El trabajo podría distribuirse de la siguiente forma:

Servidor	Puntos asignados
1	25 000 000
2	25 000 000
3	25 000 000
4	25 000 000

El cliente creará cuatro hilos y cada uno realizará simultáneamente una llamada al servidor correspondiente.

Supóngase que los servidores devuelven los siguientes resultados:

Servidor	Puntos bajo la curva
1	18 670 400
2	18 671 200
3	18 669 850
4	18 672 100

El cliente deberá calcular

$$M = M_1 + M_2 + M_3 + M_4$$

y finalmente

$$\boxed{I \approx \frac{M}{100\,000\,000}.}$$

7. Evaluación del rendimiento

Se deberán realizar experimentos utilizando diferentes números de servidores. Como mínimo se recomienda utilizar:

$$1, \quad 2, \quad 4$$

servidores y, si se dispone de ellos, también 8.

Para cada configuración se deberá utilizar el mismo número total de puntos y medir el tiempo necesario para completar el cálculo.

El alumno deberá calcular el **speedup**:

$$\boxed{S_p = \frac{T_1}{T_p}}$$

donde T_1 es el tiempo empleado con un servidor y T_p el tiempo empleado con p servidores.

También se calculará la eficiencia:

$$\boxed{E_p = \frac{S_p}{p}.}$$

Los resultados deberán presentarse en una tabla semejante a:

Servidores	Tiempo (s)	Speedup	Eficiencia
1		1	1
2			
4			
8			

El alumno deberá analizar las diferencias entre el comportamiento ideal y el observado, teniendo en cuenta, entre otros factores:

- El tiempo de cálculo.
- El coste de las comunicaciones RMI.
- La creación y sincronización de los hilos.
- El reparto de carga.
- Las diferencias de velocidad entre los servidores.

8. Cuestiones adicionales

La memoria de la práctica deberá responder razonadamente a las siguientes cuestiones:

1. ¿Por qué la aproximación mejora al aumentar el número de puntos generados?
2. ¿Por qué es necesario utilizar hilos en el cliente para que las llamadas RMI se realicen en paralelo?
3. ¿Qué ocurriría con el tiempo de ejecución si el cliente realizase las llamadas a los servidores secuencialmente?
4. ¿Es de esperar que utilizar el doble de servidores reduzca exactamente a la mitad el tiempo de ejecución? Justifique la respuesta.
5. ¿Qué parte del tiempo de ejecución puede corresponder a las comunicaciones RMI?
6. ¿Qué ocurriría si uno de los servidores dejase de responder durante la ejecución?

7. ¿Cómo podría modificarse el cliente para detectar que un servidor no responde y evitar que la aplicación quede bloqueada indefinidamente?
8. ¿Qué problemas pueden aparecer si los servidores tienen capacidades de cálculo muy diferentes?
9. ¿Por qué un reparto perfectamente equilibrado en número de puntos no garantiza necesariamente un reparto perfectamente equilibrado en tiempo de ejecución?

9. Extensión opcional: reparto dinámico

Como ampliación de la práctica, se propone modificar el programa para que el trabajo no se distribuya completamente al comienzo de la ejecución.

En lugar de asignar a cada servidor una cantidad fija de puntos, el cliente dividirá el trabajo total en bloques de tamaño configurable.

Por ejemplo, si se desean generar

$$100\,000\,000$$

puntos, podrían utilizarse bloques de

$$1\,000\,000$$

puntos.

Cuando un servidor termine un bloque, el cliente podrá asignarle otro bloque pendiente.

Esta estrategia permitirá comparar:

- Un **reparto estático**, en el que cada servidor recibe su carga al comienzo.
- Un **reparto dinámico**, en el que los servidores van recibiendo nuevos bloques a medida que terminan los anteriores.

El alumno deberá analizar las ventajas e inconvenientes de ambas estrategias, especialmente cuando los servidores tengan diferentes capacidades de cálculo.

10. Entrega

La entrega deberá incluir:

1. Código fuente completo del cliente.
2. Código correspondiente a la creación y gestión de los hilos del cliente.
3. Código fuente de los servidores.
4. Interfaces RMI.
5. Instrucciones necesarias para compilar la aplicación.
6. Instrucciones necesarias para ejecutar el registro RMI, los servidores y el cliente.
7. Memoria con los experimentos realizados.
8. Tabla de tiempos para diferentes números de servidores.

9. Cálculo del speedup y de la eficiencia.
10. Análisis de los resultados obtenidos.
11. Respuestas razonadas a las cuestiones planteadas.

Las respuestas a las cuestiones planteadas deberán adjuntarse en un documento word de nombre **memoria.doc** junto con el código fuente de la aplicación.

11. Valoración (sobre un máximo de 10)

- 5 puntos. Aplicación con un único servidor y un cliente. No es necesario elaborar una memoria.
- 8 puntos. Aplicación con múltiples servidores y un cliente con threads. Es indispensable responder a todas y cada una de las cuestiones planteadas.
- 10 puntos. Implementación de un reparto dinámico de carga entre los distintos servidores.